

# Proxy inverso y balanceador de carga

Apache, Nginx y HAProxy

✉ José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📖 SRI · Servicios de Red e Internet

# 01

## Proxy inverso

---

Concepto y problemas comunes

## ¿Qué es un proxy inverso?

*" Un **proxy inverso** acepta las peticiones de los clientes, las **reenvía** a uno o varios servidores internos (**backends**) y devuelve sus respuestas como si las hubiera generado él mismo.*

- El cliente **no accede directamente** a los backends: solo ve al proxy
- Los backends pueden estar en una **red privada** sin acceso desde fuera
- Para el backend, **el cliente es el proxy**: la conexión TCP la abre el proxy

 Un **proxy** actúa en nombre de los clientes; un **proxy inverso**, en nombre de los servidores (lo vimos en *Protocolo HTTP*).

## Casos de uso

### PROTECCIÓN Y CENTRALIZACIÓN

- **Ocultar** la infraestructura backend
- Publicar varias aplicaciones internas bajo **un único nombre o IP**
- Aplicar **filtros** y **autenticación** en un solo punto
- Centralizar las **cabeceras** de seguridad

### RENDIMIENTO Y OPERACIÓN

- **Distribuir carga** entre varios backends
- **Terminación TLS**: HTTPS solo en el proxy
- **Caché** de respuestas estáticas
- Cambiar los backends **sin que el cliente lo note**

## ¿Qué `Host` recibe el backend?

El proxy hace una **petición nueva** al backend. ¿Qué pone en la cabecera `Host` ?

| OPCIÓN                   | <code>HOST</code> QUE LLEGA AL BACKEND                                       |
|--------------------------|------------------------------------------------------------------------------|
| Por defecto              | El nombre del <b>destino del proxy</b> ( <code>interno.example1.org</code> ) |
| Conservar el del cliente | El nombre <b>público</b> que pidió el cliente ( <code>www.app1.org</code> )  |

⚠ Si el backend tiene **virtual hosts por nombre**, elige el sitio por la cabecera `Host` . Si recibe un nombre que no tiene en ningún `ServerName` , responde su **sitio por defecto**, no el que esperabas.

- Se conserva el `Host` del cliente cuando el backend **sí** tiene configurado el nombre público
- En el log del backend se ve qué virtual host ha atendido la petición

## El problema de las redirecciones

Si el backend responde con una redirección, la cabecera `Location` lleva **su nombre interno**:

```
HTTP/1.1 301 Moved Permanently
Location: http://interno.example1.org/nuevodirectorio
```

El navegador intenta ir a `interno.example1.org`, un nombre que **no resuelve** (o que no es accesible) desde fuera.

 El proxy tiene que **reescribir la cabecera** `Location` de la respuesta para que apunte al nombre público:  
`ProxyPassReverse` en Apache, `proxy_redirect` en Nginx.

## Publicar en una subruta

Publicar una aplicación en `www.ejemplo.org/shop` tiene **dos trampas**:

### LA BARRA FINAL

Si el proxy está configurado para `/shop/`, una petición a `/shop` (sin barra) **no coincide**.

Solución: redirigir `/shop` a `/shop/` (Nginx lo hace solo).

### ENLACES ABSOLUTOS

Si la página del backend enlaza `/css/estilo.css`, el navegador lo pide al proxy **sin** `/shop`: la página sale **sin estilos** o rota.

Solución: que la aplicación use enlaces relativos o permita indicar su ruta base, o **reescribir el HTML** en el proxy.

SRI · UNIDAD 2 — PROXY INVERSO Y BALANCEADOR DE CARGA

# 02

## Proxy inverso con Apache

---

mod\_proxy y ProxyPass

## Activar los módulos necesarios

Apache implementa el proxy inverso con los módulos `mod_proxy` y `mod_proxy_http`.

```
sudo a2enmod proxy proxy_http
sudo systemctl restart apache2
```

## OTROS MÓDULOS RELACIONADOS

- `mod_headers` — añadir o cambiar cabeceras de la petición y de la respuesta
- `mod_proxy_html` — reescribir los enlaces del HTML que devuelve el backend

## Configuración básica

Un virtual host por cada nombre público ( `www.app1.org` , `www.app2.org` ...):

```
<VirtualHost *:80>
  ServerName www.app1.org
  ProxyPass      "/" "http://interno.example1.org/"
  ProxyPassReverse "/" "http://interno.example1.org/"
</VirtualHost>
```

- **ProxyPass** — reenvía las peticiones al backend
- **ProxyPassReverse** — reescribe `Location` en las respuestas (redirecciones)
- El proxy tiene que **resolver** `interno.example1.org` (DNS o `/etc/hosts` )
- **ProxyRequests Off** (por defecto): con `On` , Apache sería un proxy **abierto** a Internet

## El `Host` y las redirecciones en Apache

### `PROXYPRESERVEHOST`

- `off` (por defecto): el backend recibe el nombre de `ProxyPass`  
( `interno.example1.org` )
- `On` : recibe el nombre que pidió el cliente  
( `www.app1.org` )

```
ProxyPreserveHost On
```

### `PROXYPASSREVERSE`

Cambia en la respuesta

```
http://interno.example1.org/nuevodirectorio
```

por

```
http://www.app1.org/nuevodirectorio
```

Sin esta línea, el cliente recibe el nombre interno.

## Proxy de subrutas en Apache

```
ProxyPass          "/shop/" "http://192.168.100.10:3000/"
ProxyPassReverse   "/shop/" "http://192.168.100.10:3000/"

ProxyPass          "/game/" "http://192.168.100.10:8080/"
ProxyPassReverse   "/game/" "http://192.168.100.10:8080/"

# /shop y /game sin barra no coinciden con ProxyPass
RedirectMatch "^/(shop|game)$" "/$1/"
```

- Las dos URL de `ProxyPass` tienen que acabar **igual**: las dos con barra o las dos sin ella
- Sin el `RedirectMatch`, `/shop` lo sirve el `DocumentRoot` del proxy: **404**
- Para los enlaces absolutos: `mod_proxy_html` ( `ProxyHTMLEnable On`, `ProxyHTMLURLMap` )

## Cabeceras hacia el backend

Para el backend, el cliente es el proxy. `mod_proxy_http` añade **automáticamente**:

| CABECERA                        | CONTENIDO                                                         |
|---------------------------------|-------------------------------------------------------------------|
| <code>X-Forwarded-For</code>    | IP del cliente original (se van acumulando si hay varios proxies) |
| <code>X-Forwarded-Host</code>   | <code>Host</code> que pidió el cliente                            |
| <code>X-Forwarded-Server</code> | <code>ServerName</code> del proxy                                 |

Otras cabeceras se añaden con `mod_headers` ( `sudo a2enmod headers` ):

```
RequestHeader set X-Real-IP      "expr=%{REMOTE_ADDR}"
RequestHeader set X-Forwarded-Proto "http"
```

# 03

## Proxy inverso con Nginx

---

proxy\_pass y proxy\_params

## Configuración básica

Nginx incluye el proxy inverso de forma **nativa**, sin activar módulos.

```
server {  
    listen 80;  
    server_name www.app1.org;  
  
    location / {  
        proxy_pass http://interno.example1.org/;  
    }  
}
```

- **proxy\_pass** — destino al que se reenvía la petición
- Si Nginx **no resuelve** el nombre de **proxy\_pass**, no arranca ( **host not found in upstream** )

## El `Host` en Nginx

Por defecto Nginx envía `Host: $proxy_host`, el nombre de `proxy_pass` (`interno.example1.org`).

Para enviar el que pidió el cliente:

```
location / {  
    proxy_pass      http://interno.example1.org/;  
    proxy_set_header Host $host;  
}
```

⚠ `include proxy_params` también cambia el `Host` por el del cliente. Si el backend tiene virtual hosts con los **nombres internos**, responderá su sitio por defecto.

## El archivo `proxy_params`

`/etc/nginx/proxy_params` (Debian) envía al backend la información del cliente original:

```
proxy_set_header Host          $http_host;
proxy_set_header X-Real-IP     $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

| VARIABLE                                 | CONTENIDO                                                                             |
|------------------------------------------|---------------------------------------------------------------------------------------|
| <code>\$http_host</code>                 | <code>Host</code> que pidió el cliente                                                |
| <code>\$proxy_add_x_forwarded_for</code> | <code>X-Forwarded-For</code> recibido + IP del cliente ( <code>\$remote_addr</code> ) |

A diferencia de Apache, Nginx **no** añade estas cabeceras si no se indican.

## Reescritura de redirecciones

Nginx **ya reescribe** la cabecera `Location` por defecto ( `proxy_redirect default` ): cambia la URL de `proxy_pass` por la de la `location` y la completa con el nombre que pidió el cliente.

```
location / {  
    proxy_pass      http://interno.example1.org/;  
    # proxy_redirect default;    ← es el comportamiento por defecto  
    # proxy_redirect off;        ← no reescribe: el cliente recibe el nombre interno  
}
```

### RESPUESTA DEL BACKEND

Location: http://interno.example1.org/nuevodirectorio

### LO QUE RECIBE EL CLIENTE

Location: http://www.app1.org/nuevodirectorio

Equivale al `ProxyPassReverse` de Apache, que en cambio hay que escribir siempre.

## Proxy de subrutas en Nginx

```
server {  
    listen 80;  
    server_name www.ejemplo.org;  
    location /shop/ { proxy_pass http://192.168.100.10:3000/; }  
    location /game/ { proxy_pass http://192.168.100.10:8080/; }  
}
```

- **Barra final en `proxy_pass`**: con `/` al final se quita el prefijo ( `/shop/a` → `/a` ); sin barra, se reenvía la URL completa ( `/shop/a` )
- Si la `location` acaba en `/`, Nginx redirige solo `/shop` a `/shop/`
- Para los enlaces absolutos: `sub_filter` (reescribe el texto del HTML)

## Reescribir los enlaces del HTML

Si el backend usa enlaces absolutos ( `/css/...` ), el proxy puede añadirles el prefijo:

**Apache** ( `a2enmod proxy_html xml2enc` )

```
<Location "/shop/">
  ProxyPass "http://192.168.100.10:3000/"
  ProxyPassReverse "http://192.168.100.10:3000/"
  ProxyHTMLEnable On
  ProxyHTMLURLMap "/" "/shop/"
</Location>
```

**Nginx** ( `sub_filter` )

```
location /shop/ {
  proxy_pass http://192.168.100.10:3000/;
  proxy_set_header Accept-Encoding "";
  sub_filter 'href=/' 'href="/shop/';
  sub_filter 'src=/' 'src="/shop/';
  sub_filter_once off;
}
```

⚠ Solo cambia el **HTML**, no el JavaScript ni el CSS. Si la aplicación no funciona en una subruta, se publica con su **propio nombre** ( `shop.ejemplo.org` ).

## Apache vs Nginx — proxy inverso

| ASPECTO                       | APACHE                                          | NGINX                                                       |
|-------------------------------|-------------------------------------------------|-------------------------------------------------------------|
| Módulos                       | <code>a2enmod proxy proxy_http</code>           | Nativo                                                      |
| Reenvío                       | <code>ProxyPass</code>                          | <code>proxy_pass</code>                                     |
| Redirecciones                 | <code>ProxyPassReverse</code> (hay que ponerlo) | <code>proxy_redirect default</code> (activo)                |
| <code>Host</code> por defecto | El de <code>ProxyPass</code>                    | El de <code>proxy_pass</code> ( <code>\$proxy_host</code> ) |
| <code>Host</code> del cliente | <code>ProxyPreserveHost On</code>               | <code>proxy_set_header Host \$host</code>                   |
| <code>X-Forwarded-For</code>  | Automática                                      | <code>proxy_params</code> O <code>proxy_set_header</code>   |
| Subruta sin barra             | <code>RedirectMatch</code>                      | Redirección automática                                      |
| Comprobar sintaxis            | <code>apache2ctl configtest</code>              | <code>nginx -t</code>                                       |

SRI · UNIDAD 2 — PROXY INVERSO Y BALANCEADOR DE CARGA

# 04

## **Balanceo de carga**

---

Algoritmos y software

## Algoritmos de balanceo — estáticos

No tienen en cuenta el estado de los servidores: el reparto se decide **de antemano**.

### ROUND ROBIN

Las peticiones se reparten **por turnos** entre los servidores. Requiere servidores equivalentes.

### ROUND ROBIN PONDERADO

Cada servidor tiene un **peso**: los más potentes reciben más peticiones.

### STICKY (AFINIDAD)

Un mismo cliente va **siempre al mismo servidor** (por su IP o con una cookie).

### HASH

Una función *hash* sobre la IP, la URL... **determina** qué servidor atiende cada petición.

## Algoritmos de balanceo — dinámicos

Tienen en cuenta el **estado real** de los servidores en cada momento.

### CONEXIONES MÍNIMAS

La nueva petición va al servidor con **menos conexiones activas**.

### MENOR TIEMPO DE RESPUESTA

Se elige el servidor con el **tiempo de respuesta más bajo** según las últimas mediciones.

**i** Se adaptan mejor a peticiones de duración muy distinta, pero el balanceador tiene que **medir** el estado de cada servidor.

## Software de balanceo de carga

| SOFTWARE                                          | CARACTERÍSTICAS                                                                                      |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>HAProxy</b>                                    | Especializado en balanceo, TCP y HTTP. <i>Health checks</i> activos y página de estadísticas         |
| <b>Nginx</b> ( <code>upstream</code> )            | Servidor web que también balancea: bloque <code>upstream</code> + <code>proxy_pass</code>            |
| <b>Apache</b> ( <code>mod_proxy_balancer</code> ) | <code>&lt;Proxy "balancer:// ... "&gt;</code> CON <code>BalancerMember</code>                        |
| <b>Traefik</b>                                    | Proxy inverso para <b>contenedores</b> (Docker, Kubernetes): se configura solo a partir de etiquetas |

 También hay balanceadores **hardware** y los que ofrecen los proveedores de nube. En esta unidad usamos **HAProxy**.

# 05

## **Balanceador de carga con HAProxy**

---

Distribución de tráfico entre backends

## ¿Qué es HAProxy?

" *HAProxy (High Availability Proxy) es **software libre de alto rendimiento** especializado en **balanceo de carga** y **proxy inverso** para servicios **TCP** y **HTTP**.*

- **Estabilidad** y rendimiento muy contrastados
- **Health checks** automáticos sobre los backends
- Varios **algoritmos** de balanceo
- Puede balancear **TCP genérico** o entender **HTTP** (cabeceras, cookies)
- Página de **estadísticas** para ver y operar el balanceador

# Escenario de laboratorio

| MÁQUINA     | ROL          | RED EXTERNA   | RED DE DATOS    |
|-------------|--------------|---------------|-----------------|
| balanceador | HAProxy      | 192.168.10.10 | 192.168.100.1   |
| apache1     | Servidor web | 192.168.10.11 | 192.168.100.100 |
| apache2     | Servidor web | 192.168.10.12 | 192.168.100.101 |
| Tu equipo   | Cliente      | 192.168.10.1  | —               |

- El cliente accede a `www.example.org`, que apunta a la IP **externa del balanceador**
- HAProxy se conecta a los servidores web por la **red de datos**
- Cada servidor web muestra su nombre, para ver quién atiende cada petición

# Instalación

```
sudo apt install haproxy
systemctl status haproxy

# Validar la sintaxis antes de aplicar cambios
sudo haproxy -c -f /etc/haproxy/haproxy.cfg

# Aplicar cambios
sudo systemctl reload haproxy
```

Los mensajes de HAProxy van al journal: `journalctl -u haproxy`.

# Estructura del archivo de configuración

`/etc/haproxy/haproxy.cfg` se organiza en secciones:

| SECCIÓN               | PARA QUÉ SIRVE                                                                              |
|-----------------------|---------------------------------------------------------------------------------------------|
| <code>global</code>   | Parámetros del demonio (logs, usuario, límites)                                             |
| <code>defaults</code> | Valores por defecto para <code>frontend</code> , <code>backend</code> y <code>listen</code> |
| <code>frontend</code> | Cómo se <b>reciben</b> las peticiones (IP, puerto)                                          |
| <code>backend</code>  | A qué <b>servidores</b> se reenvían y con qué algoritmo                                     |
| <code>listen</code>   | <code>frontend</code> + <code>backend</code> en un solo bloque                              |

 El fichero que instala Debian ya trae `global` y `defaults` : solo hay que añadir el `frontend` y el `backend` .

## Secciones `global` y `defaults`

```
global
  log /dev/log local0
  user haproxy
  group haproxy
```

```
defaults
  log      global
  mode     http
  option   httplog
  timeout  connect 5s
  timeout  client  50s
  timeout  server  50s
```

- `mode http` — entiende HTTP (alternativa: `mode tcp` para TCP genérico)
- `timeout` — tiempos de espera de conexión, cliente y servidor

## Secciones `frontend` y `backend`

```
frontend servidores_web
  bind *:80
  default_backend servidores_web_backend

backend servidores_web_backend
  balance roundrobin
  option httpchk GET /
  server apache1 192.168.100.100:80 check
  server apache2 192.168.100.101:80 check
```

- `bind *:80` — escucha en el puerto 80 · `default_backend` — adónde van las peticiones
- `balance` — algoritmo de balanceo · `server ... check` — con *health check* activo

# Algoritmos en HAProxy

| BALANCE    | ALGORITMO                                        | TIPO     |
|------------|--------------------------------------------------|----------|
| roundrobin | Round Robin (ponderado con <code>weight</code> ) | Estático |
| source     | Hash de la IP del cliente: afinidad por IP       | Estático |
| uri        | Hash de la URL: la misma URL, al mismo servidor  | Estático |
| leastconn  | Conexiones mínimas                               | Dinámico |

```
server apache1 192.168.100.100:80 check weight 2
server apache2 192.168.100.101:80 check weight 1
```

## Health checks

HAProxy pide `GET /` a cada servidor cada cierto tiempo (vale un 2xx o 3xx):

```
backend servidores_web_backend
  option httpchk GET /
  server apache1 192.168.100.100:80 check inter 2s fall 3 rise 2
  server apache2 192.168.100.101:80 check inter 2s fall 3 rise 2
```

| PARÁMETRO          | SIGNIFICADO                                        |
|--------------------|----------------------------------------------------|
| <code>inter</code> | Intervalo entre comprobaciones                     |
| <code>fall</code>  | Fallos seguidos para marcarlo <b>DOWN</b>          |
| <code>rise</code>  | Aciertos seguidos para volver a marcarlo <b>UP</b> |

Si uno falla, **deja de enviarle tráfico**. Si caen todos: **503**.

## Acceso desde el cliente

En el `/etc/hosts` del cliente: `192.168.10.10 www.example.org`

Varias peticiones seguidas:

```
$ for i in 1 2 3; do curl -s http://www.example.org/ | grep Servidor; done  
<h1>Servidor: apache1</h1>  
<h1>Servidor: apache2</h1>  
<h1>Servidor: apache1</h1>
```

Con `roundrobin`, las respuestas **alternan** entre los dos servidores.

## La IP del cliente en los servidores web

Para el servidor web, el cliente es el **balanceador**:

```
192.168.100.1 - - [03/Oct/2026:10:15:02 +0200] "GET / HTTP/1.1" 200 312 "-" "curl/8.14.1"
```

Con `option forwardfor`, HAProxy añade la cabecera `X-Forwarded-For` con la IP real del cliente:

```
backend servidores_web_backend
    option forwardfor
```

 La cabecera llega al servidor web, pero el log **no la registra** hasta que se cambia su formato.

## Registrar la IP real en el log

```
# Apache
LogFormat "%{X-Forwarded-For}i %l %u %t \"%r\" %>s %0" balanceado
CustomLog ${APACHE_LOG_DIR}/access.log balanceado
```

```
# Nginx
log_format balanceado '$http_x_forwarded_for - [$time_local] "$request" $status';
access_log /var/log/nginx/access.log balanceado;
```

### Resultado:

```
192.168.10.1 - - [03/Oct/2026:10:16:40 +0200] "GET / HTTP/1.1" 200 312
```

## Afinidad de sesión: el problema

Una **sesión** de PHP se guarda **en el servidor** ( `/var/lib/php/sessions` ); el navegador solo guarda la cookie con el identificador ( `PHPSESSID` ).



Con `roundrobin`, la siguiente petición va a `apache2`, que **no tiene** esa sesión: crea una nueva y el contador vuelve a empezar.

## Afinidad de sesión con una cookie

HAProxy añade su **propia cookie** con el servidor que ha atendido al cliente:

```
backend servidores_web_backend
  balance roundrobin
  cookie SERVERID insert indirect nocache
  server apache1 192.168.100.100:80 check cookie s1
  server apache2 192.168.100.101:80 check cookie s2
```

- **insert** — HAProxy envía `Set-Cookie: SERVERID=s1`; el navegador la devuelve en cada petición y HAProxy lo manda a `apache1`
- **indirect** — la cookie no llega al servidor web · **nocache** — que no la guarden las cachés



Alternativas: `balance source` (por IP) o sesiones **compartidas** entre servidores (base de datos, Redis).

## Comprobar la afinidad

`curl` **no guarda cookies**: cada petición es un cliente nuevo y el contador siempre vale 1.

```
$ curl -sI http://www.example.org/sesion.php | grep -i set-cookie
set-cookie: PHPSESSID=k3j9q2...; path=/
set-cookie: SERVERID=s1; path=/
```

Para que `curl` guarde las cookies y las envíe en las siguientes peticiones:

```
$ curl -s -c cookies.txt -b cookies.txt http://www.example.org/sesion.php
```

Repetido varias veces: **siempre el mismo servidor** y el contador sube. El navegador lo hace solo.

## Página de estadísticas

Bloque `listen` propio, en otro puerto ( `http://192.168.10.10:8404/stats` ):

```
listen estadisticas
  bind *:8404
  stats enable
  stats uri /stats
  stats auth admin:asir
  stats admin if TRUE
```

- Estado de cada servidor (**UP** / **DOWN** / **MAINT**), sesiones y errores
- Con `stats admin`, poner un servidor en **mantenimiento** desde la web

 Siempre con autenticación y nunca accesible desde redes públicas.

## Proxy inverso y balanceador juntos



- **Proxy inverso:** único punto de entrada; elige el destino por el **nombre** o la **ruta**
- **Balanceador:** reparte entre servidores **que sirven lo mismo** y retira los que fallan
- Para el proxy, HAProxy es **un backend más**; para HAProxy, el cliente es el proxy
- X-Forwarded-For **acumula** las IP: X-Forwarded-For: <cliente>, <proxy>

SRI · UNIDAD 2 — PROXY INVERSO Y BALANCEADOR DE CARGA

# 06

## Resumen

---

Problemas frecuentes

## Problemas frecuentes: proxy inverso

| SÍNTOMA                                  | CAUSAS HABITUALES                                                                     |
|------------------------------------------|---------------------------------------------------------------------------------------|
| <b>502</b> Bad Gateway                   | El backend no responde: servicio parado, IP o puerto mal                              |
| <b>504</b> Gateway Timeout               | El backend tarda más que el <code>timeout</code> del proxy                            |
| <b>Sale otro sitio</b>                   | El <code>Host</code> que recibe el backend no coincide con su <code>ServerName</code> |
| <b>Redirige a un nombre interno</b>      | Falta <code>ProxyPassReverse</code> o hay <code>proxy_redirect off</code>             |
| <b>Página sin estilos</b> en una subruta | Enlaces absolutos del backend · falta la barra final                                  |
| <b>Nginx no arranca</b>                  | No resuelve el nombre de <code>proxy_pass</code>                                      |

Probar sin tocar `/etc/hosts`: `curl -H "Host: www.app1.org" http://<ip del proxy>/`

Antes de recargar: `apache2ctl configtest` o `nginx -t`. Ante un error: los logs del **proxy** y del **backend**.

## Problemas frecuentes: balanceador

| SÍNTOMA                                                      | CAUSAS HABITUALES                                                                                           |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>503</b> Service Unavailable                               | Todos los servidores <b>DOWN</b> : servicios parados o el <i>health check</i> pide una página que no existe |
| En el log del servidor web sale la <b>IP del balanceador</b> | Falta <code>option forwardfor</code> o cambiar el formato del log                                           |
| El contador de la sesión <b>vuelve a 1</b>                   | Sin afinidad · o se prueba con <code>curl</code> sin guardar cookies                                        |
| Siempre responde <b>el mismo servidor</b>                    | El otro está DOWN · afinidad activa · <code>balance source</code>                                           |

Antes de recargar: `haproxy -c -f /etc/haproxy/haproxy.cfg`. El estado de cada servidor, en la página de **estadísticas**.

## Para profundizar

- [Documentación oficial de HAProxy](#)
- [Apache mod\\_proxy](#)
- [Nginx — proxy\\_pass](#)

SRI · UNIDAD 2 — PROXY INVERSO Y BALANCEADOR DE CARGA

# ¡Gracias!

Proxy inverso y balanceo

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI